Spectre Without Dependent Load

Can Aknesil[1] Andreas Lindner[2] Roberto Guanciale[1] Hamed Nemati[1]

[1]KTH Royal Institute of Technology, Stockholm, Sweden
{aknesil, robertog, hnnemati}@kth.se

[2]Uppsala University, Uppsala, Sweden
andreas.lindner@angstrom.uu.se

***Abstract*—Transient execution attacks that disclose arbitrary memory commonly assume a multi-stage *read-then-transmit* gadget: a transient load to fetch secret data and a subsequent operation to leak that data into an observable side channel. We show that this assumption does not hold under electromagnetic (EM) observations, by verifying that a single transient load already produces value-dependent EM leakage without any explicit follow-up transmission instruction or relying on prefetching. Our results expand the set of exploitable gadgets and show that even simple processors like the Cortex-A53 are vulnerable.**

Keywords—microarchitectural security, electromagnetic side channel attacks, speculative execution

I. Introduction

Practical attacks that leverage speculative execution were first demonstrated in 2018 with the infamous Spectre attack [1] that exploits transient instructions to extract arbitrary information from a target program's memory.

In this paper, we show that transient execution attacks that leak arbitrary memory are possible with simpler gadgets than demonstrated before, given that the adversary can observe the EM side channel. Such gadgets can be executed on processors with limited speculation support that were previously considered secure, e.g., ARM Cortex-A53. Additionally, the high prevalence of simpler gadgets in software makes existing mitigations ineffective and increases the performance overhead of protected programs. The vulnerability we identified has not been reported in prior work.

We confirm the discovered vulnerability on Raspberry Pi 3, featuring an ARM Cortex-A53 processor, through Test Vector Leakage Assessment (TVLA).

II. Relaxing the Spectre-PHT Gadget

The gadget used in the Spectre PHT variant, namely the *double-load gadget*, is shown in Fig. 1. The vast majority of transient execution attacks consists of three distinct components: (line 1) an instruction whose result is predicted by the processor, in this case, the result of a conditional branch; (line 2) a reading instruction that allows the adversary to speculatively fetch from attacker-controlled addresses; (line 3) a transmitting instruction that affects the microarchitectural state with the fetched value.

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, the Wallenberg Launch Pad (WALP), and a gift from Intel and Amazon.

Fig. 1: Double-load gadget, i.e., the original Spectre gadget (top), single-load gadget (bottom).

```
1 if (x < A_size) {
2   y = A[x];         // Load 1
3   z = B[y];         // Load 2
4 }
```

```
1 if (x < A_size) {
2   y = A[x];         // Load 1
3 }
```

On a non-speculative processor, the program cannot read A out of bounds, and the only values that can be exfiltrated via a cache side channel are x and the elements of A. This program would be considered free of side-channel vulnerabilities as long as both x and A are public. However, a speculative processor can transiently execute the conditional block, even if the index x is out of bounds, reading and processing arbitrary values from the program's memory. In case of misprediction, the calculations done during speculation are discarded; however, the side effects may not be completely reversed, leaving traces of the information read in various side channels. In this case, line 3 would leak y by affecting the cache. Similarly, processing of y can also leave traces in the power and EM side channels.

The *single-load gadget*, shown in Fig. 1, is a relaxed version of the double-load gadget, where Load 2 is not present, or not executed in speculation. To date, the single-load gadget has been considered less dangerous than the double-load gadget, as only reading arbitrary memory is assumed to be harmless without any subsequent instruction that processes and, thus, leaks the resulting value. In this paper, however, we show that a single load instruction can read an out-of-bounds value from the memory and, simultaneously, leak it into the EM side channel. This is simply because loading data from memory has an associated footprint in the EM radiation. From this realization, it follows that the single-load gadget can be as dangerous as the double-load gadget, given that the adversary can observe the EM side channel.

Due to the relaxed requirements of the single-load gadget compared to the double-load gadget, some processors can speculatively execute the former but not the latter. An example is the Cortex-A53 processor, which can execute load operations in speculation, but cannot if they depend on speculatively loaded values [2]. Another example is the CVA6 RISC-V processor [3], which can only execute a single load operation

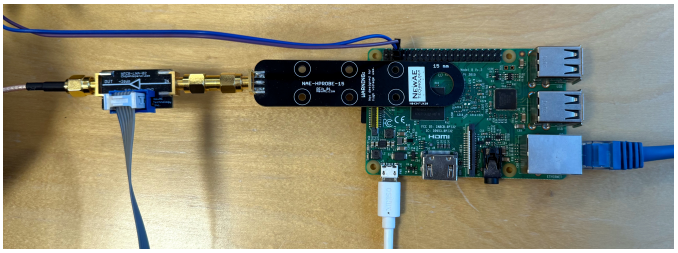

Fig. 2: EM side-channel analysis setup showing the near-field EM probe connected to the signal amplifier placed on the target device.

in speculation if configured with a small scoreboard.

III. Experimental Setup

The experimental setup, shown in Fig. 2, consists of a Raspberry Pi 3 Model B Rev 1.2 [4] with a Cortex-A53 processor, where the target program is executed, a CW1173 ChipWhisperer Lite capture board [5], a NAE-HPROBE-15 near-field EM probe [6], and a CW502 amplifier [7].

The target program contains a single-load gadget that leaks a 4-byte word through speculatively loading from the main memory. It is executed as a regular user-space process on the Linux OS using the default configuration provided for the Raspberry Pi 3, and therefore runs concurrently with other processes on the system. Although we did not run any resource-intensive workloads that could significantly interfere with the target program, the target program is still subject to the typical background activity present in a near-idle system.

The capture device is configured with a sampling frequency of 105 MHz (maximum possible for the capture device). Capturing is synchronized with the execution of the target program via an external trigger signal; however, without clock synchronization. All measurements are performed in a typical office environment without any protection against environmental noise.

IV. Evaluation

The evaluation comprises Test Vector Leakage Assessment (TVLA). The results prove that the resulting value of a transient load instruction leaks into the EM side channel.

TVLA is performed byte-wise using *Welch's t-test* [8] according to the Hamming weight (number of 1s) of the leaking value. The number of traces used in TVLA is 3,600,000.

TVLA results are illustrated alongside the averaged trace in Fig. 3. The maximum t-value magnitudes for each byte, from 0 to 3, of the leaking word are 110.6, 170.2, 61.5, and 38.8, respectively. The results strongly indicate the presence of leakage, as all t-value magnitudes are significantly greater than 4.5.

V. Conclusion

We have uncovered a new attack vector related to EM leakage due to speculative execution by showing that the transient execution of a single load instruction can leak arbitrary memory through the EM side channel. Due to the simplicity of the single-load gadget, the attack is possible on simpler hardware that was previously considered secure, and countermeasures

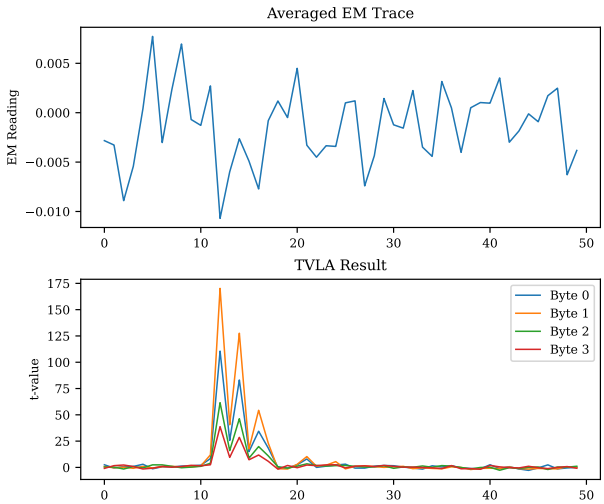

Fig. 3: Averaged EM trace (top) and the TVLA results (bottom).

needed for mitigating the presented vulnerability are expected to be costlier than existing ones.

Although the experiments focus on conditional branch prediction, it is evident that the same vulnerability also applies to other forms of speculation. Moreover, the attack is likely to be feasible through additional side channels, in particular, power side channels, whose physical characteristics are closely related to those of EM leakage.

Responsible Disclosure

After confirming the vulnerability experimentally, we contacted ARM and Raspberry Pi, and neither raised any objections to publishing our findings.

References

[1] P. Kocher, J. Horn, A. Fogh, , D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.

[2] P. Buiras, H. Nemati, A. Lindner, and R. Guanciale, "Validation of side-channel models via observation refinement," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 578–591. [Online]. Available: https://doi.org/10.1145/3466752.3480130

[3] F. Zaruba and L. Benini, "The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, Nov 2019.

[4] Accessed: Jan. 12, 2026. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-3-model-b/

[5] "CW1173 ChipWhisperer-Lite," accessed: Jan. 12, 2026. [Online]. Available: https://chipwhisperer.readthedocs.io/en/latest/Capture/ChipWhisperer-Lite.html

[6] Accessed: Jan. 12, 2026. [Online]. Available: https://media.newae.com/datasheets/NAE-HPROBE-15_datasheet.pdf

[7] "CW502 low noise amplifier," accessed: Jan. 12, 2026. [Online]. Available: https://chipwhisperer.readthedocs.io/en/latest/Tools/CW502%20Low%20Noise%20Amplifier.html

[8] T. Schneider and A. Moradi, "Leakage assessment methodology," in *Cryptographic Hardware and Embedded Systems – CHES 2015*, T. Güneysu and H. Handschuh, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 495–513.